Pembagian Pengerjaan Tugas 2 (UTS) Sistem Terdistribusi

Message Passing, Remote Method Invocation, dan Publish-Subscribe. Tahap awal: dikerjakan masing-masing di localhost, sebelum pengujian dua perangkat.

1. Kesepakatan Awal (15 menit, lewat chat)

Supaya hasil bisa digabung tanpa konflik, sepakati hal berikut terlebih dahulu:

- **Bahasa:** Python 3.
- **IP dan port** dijadikan variabel di atas file atau argumen `--host`, jangan di-hardcode. Pindah ke 2 perangkat nanti cukup ganti satu baris.
- **Data uji:** string/byte acak berukuran 200, 400, 600, 800, 1000 KB.
- **Port:** message passing 5000, RMI 5001 (Pyro5 nameserver 9090), MQTT 1883.
- **Struktur folder** yang sama (lihat di bawah).

```
UTS/
|-- message_passing/   (server.py, client.py)
|-- rmi/               (server.py, client.py)
|-- pubsub/            (publisher.py, subscriber.py)
|-- common/            (generator data 200-1000 KB)
`-- hasil/             (screenshot, tabel, pcap)
```

2. Ringkasan Pembagian

	Orang A	Orang B
Implementasi	Message passing, lalu RMI (Pyro5)	Pub-sub (MQTT + Mosquitto)
Tahap lanjutan	Menyelesaikan RMI	Persiapan pengujian: Wireshark, generator data, template tabel QoS
Hasil wajib	Kode jalan di localhost, screenshot terminal client dan server	Pub-sub jalan di localhost, 1 capture Wireshark contoh, generator data

3. Orang A: Message Passing lalu RMI

Tahap 1: Message passing (socket TCP)

1. `server.py`: buka socket, `listen`, terima data sampai selesai, balas "OK".
2. `client.py`: connect, kirim data berukuran tertentu, tunggu balasan.
3. Tambahkan argumen ukuran data (200-1000 KB) dan argumen `--delay` di server untuk sleep() sebelum membalas (1, 5, 10 detik).
4. Tes di localhost: kirim kelima ukuran, pastikan tidak terpotong. TCP itu stream, jadi kirim panjang data di awal pesan.

Tahap 2: RMI (Pyro5)

1. Install: `pip install Pyro5`.
2. Server: buat class dengan method, misalnya `terima_data(teks)`, daftarkan dengan `@Pyro5.api.expose`.
3. Client: panggil method itu seperti fungsi biasa.
4. Tambahkan sleep yang bisa diatur di method server.

4. Orang B: Pub-Sub (MQTT) lalu Persiapan Pengujian

Tahap 1: Pub-sub

1. Install broker: `sudo apt install mosquitto`, lalu jalankan servicenya.
2. Install library: `pip install paho-mqtt`.
3. `subscriber.py`: subscribe ke topik, tampilkan pesan masuk.
4. `publisher.py`: publish data berbagai ukuran. Cek batas ukuran pesan jika 1000 KB gagal.
5. Coba QoS 0 dan 1, lihat bedanya.
6. Tambahkan sleep di subscriber untuk uji sync/async.

Tahap 2: Persiapan pengujian (bisa di localhost)

1. Install Wireshark, latihan capture di interface loopback (`lo`).
2. Buat `common/generator.py` untuk data 200-1000 KB.
3. Latihan filter: `tcp.port == 5000`, `mqtt`; baca Statistics > Conversations dan Capture File Properties.
4. Siapkan template tabel QoS, tambahkan kolom jitter (diminta soal, tidak ada di template).

5. Catatan Penting Soal Localhost

- Wireshark di Linux menangkap traffic localhost di interface `lo`. Di Windows perlu Npcap dengan opsi loopback.
- **Nilai QoS dari localhost tidak valid** untuk laporan akhir (delay mendekati nol, tanpa packet loss). Pengujian QoS resmi harus diulang di 2 perangkat.
- Untuk marshaling (lihat payload) dan sync/async (lihat jeda), hasil localhost cukup untuk latihan.

6. Tahap Bertemu (Pengujian 2 Perangkat)

1. Gabungkan kode ke satu folder.
2. Ganti IP server di tiap client (satu baris).
3. Buka port di firewall (`sudo ufw allow 5000`, dst.).
4. Jalankan 3 pengujian resmi (QoS, marshaling, sync/async) dan isi laporan bersama.

7. Bahan untuk Report Progress

- Orang A: message passing selesai, RMI sedang dikerjakan.
- Orang B: pub-sub selesai, Wireshark dan generator siap.
- Lampirkan 2-3 screenshot terminal dan 1 screenshot Wireshark loopback.